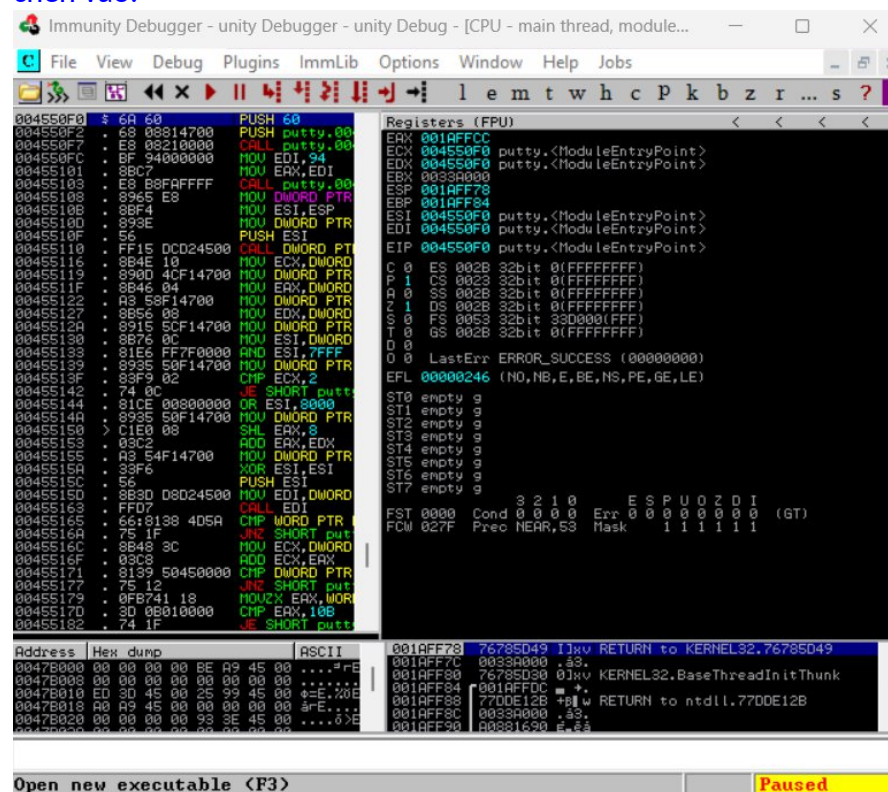


Bài Lab 17

Proj 8a: Simple EXE Hacking with Immunity

Đây là công cụ dùng để phân tích luồng thực thi của chương trình. Tại địa chỉ 0041CB6E, bạn đang xác định "điểm neo" (hook point) để bắt chương trình dừng lại và chuyển hướng thực thi từ các lệnh gốc của putty.exe sang đoạn mã độc mà bạn sẽ chèn vào.



The screenshot shows the Immunity Debugger interface. The main window displays assembly code for the 'putty.exe' module. The code is at address 004550F0 and includes instructions like PUSH, CALL, MOV, and CMP. The right-hand pane shows the 'Registers (FPU)' window, which lists various registers and their values. The 'LastErr' register is set to 'ERROR_SUCCESS (00000000)'. The bottom status bar indicates the debugger is 'Paused'.

```
004550F0  6A 60      PUSH 60
004550F2  68 08814700  PUSH putty.00
004550F7  E8 08210000  CALL putty.00
004550FC  BF 34000000  MOV EDI,34
00455101  8BC7       MOV EAX,EDI
00455103  E8 B8FAFFFF  CALL putty.00
00455108  8965 E8     MOV DWORD PTR [ESI],ESP
0045510B  8BF4       MOV ESI,ESP
0045510D  56         PUSH ESI
0045510F  FF15 DC024500  CALL DWORD PTR [EBP+2]
00455116  8B4E 10     MOV ECX,DWORD PTR [ESI+10]
00455119  8900 4CF14700  MOV DWORD PTR [EAX],ECX
0045511F  8B46 04     MOV EAX,DWORD PTR [ESI+4]
00455122  A3 58F14700  MOV DWORD PTR [EAX],EAX
00455127  8B56 08     MOV EDX,DWORD PTR [ESI+8]
0045512A  9915 5CF14700  MOV DWORD PTR [EAX],EDX
00455130  8B76 0C     MOV ESI,DWORD PTR [ESI+0C]
00455133  31E6 FF7F0000  XOR ESI,ESI
00455139  8935 50F14700  MOV DWORD PTR [EAX],ESI
0045513F  83F9 02     CMP ECX,2
00455142  74 0C      JE SHORT putty.00
00455144  81CE 00000000  OR ESI,00000000
0045514A  8935 50F14700  MOV DWORD PTR [EAX],ESI
00455150  C1E0 08     SHL EAX,8
00455153  03C2       ADD EAX,EDX
00455155  A3 54F14700  MOV DWORD PTR [EAX],EAX
0045515A  33F6       XOR ESI,ESI
0045515C  56         PUSH ESI
0045515D  8B80 D8024500  MOV EDI,DWORD PTR [EBP+2]
00455163  FFD7       CALL EDI
00455165  66:8138 405A  CMP WORD PTR [ESI+38],405A
0045516A  75 1F      JNC SHORT putty.00
0045516C  8B48 3C     MOV ECX,DWORD PTR [ESI+3C]
0045516F  83C8       ADD ECX,EAX
00455171  8139 50450000  CMP DWORD PTR [EAX],ECX
00455177  75 12      JNC SHORT putty.00
00455179  0FB741 18   MOVZX EAX,MWORD PTR [ESI+18]
0045517D  3D 0B010000  CMP EAX,10B
00455182  74 1F      JE SHORT putty.00
```

Registers (FPU)

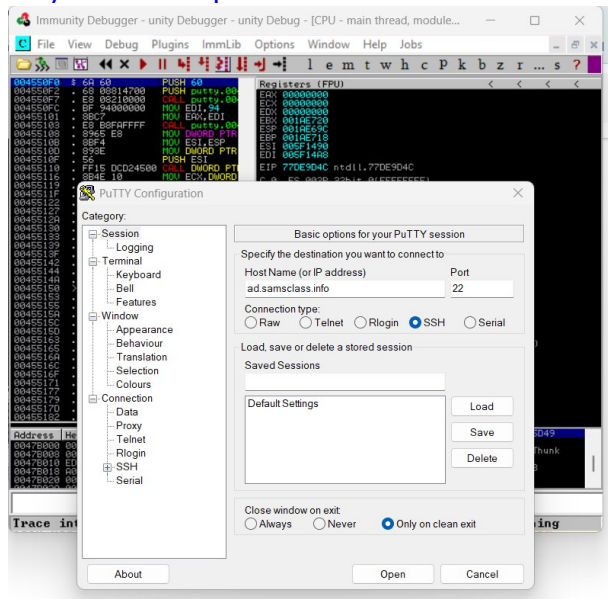
Register	Value
EAX	001AFFCC
ECX	004550F0 putty.<ModuleEntryPoint>
EDX	004550F0 putty.<ModuleEntryPoint>
EBX	0033A000
ESP	001AFF78
EBP	001AFF84
ESI	004550F0 putty.<ModuleEntryPoint>
EDI	004550F0 putty.<ModuleEntryPoint>
EIP	004550F0 putty.<ModuleEntryPoint>

Address Hex dump ASCII

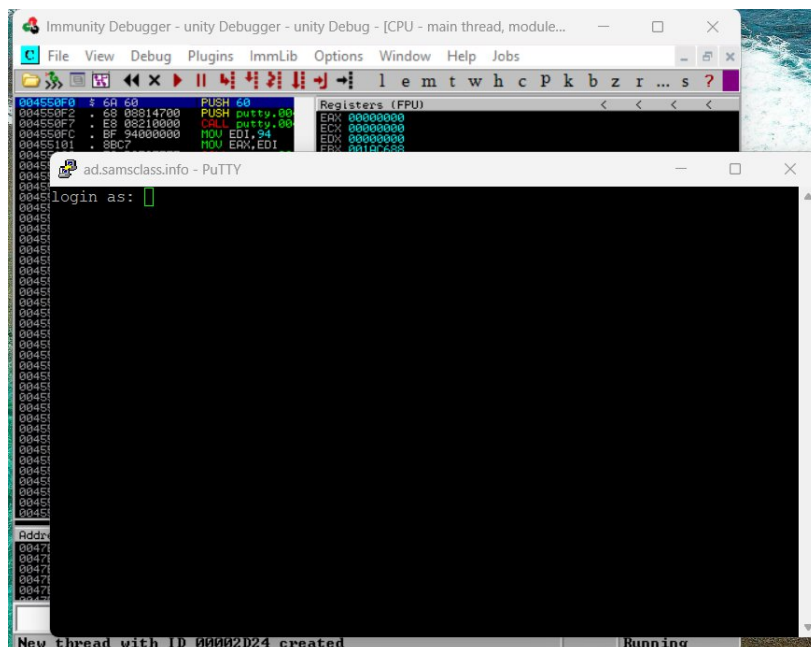
Address	Hex dump	ASCII
0047B000	00 00 00 00 BE A9 45 00	#FE
0047B008	00 00 00 00 00 00 00 00	
0047B010	ED 30 45 00 25 92 45 00	...E...%
0047B018	A0 A9 45 00 00 00 00 00	...aE...
0047B020	00 00 00 00 3E 45 00 00	>E
0047B028	00 00 00 00 00 00 00 00	

Open new executable <F3> Paused

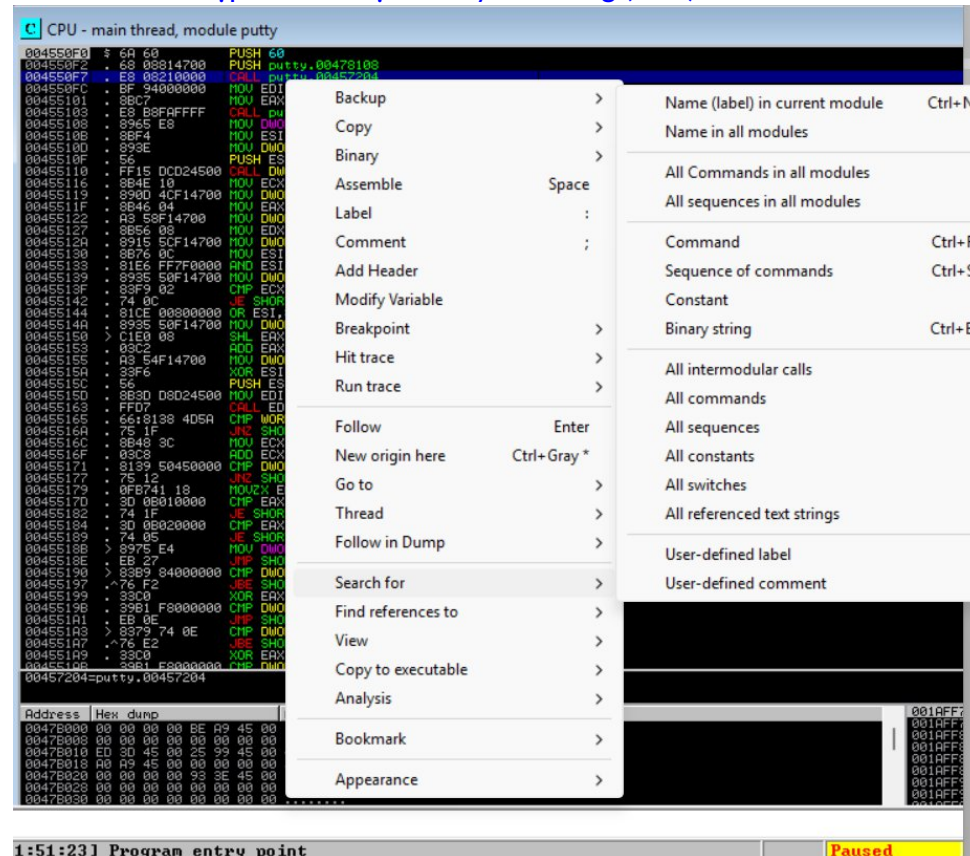
Thực hành kỹ thuật backdoor file putty.exe bằng cách thêm section mới, chuyển hướng luồng thực thi sang shellcode và chèn mã độc mở cổng 4444. Kết quả là file vẫn chạy bình thường nhưng âm thầm mở cổng kết nối, cho phép hacker điều khiển máy tính từ xa qua telnet.



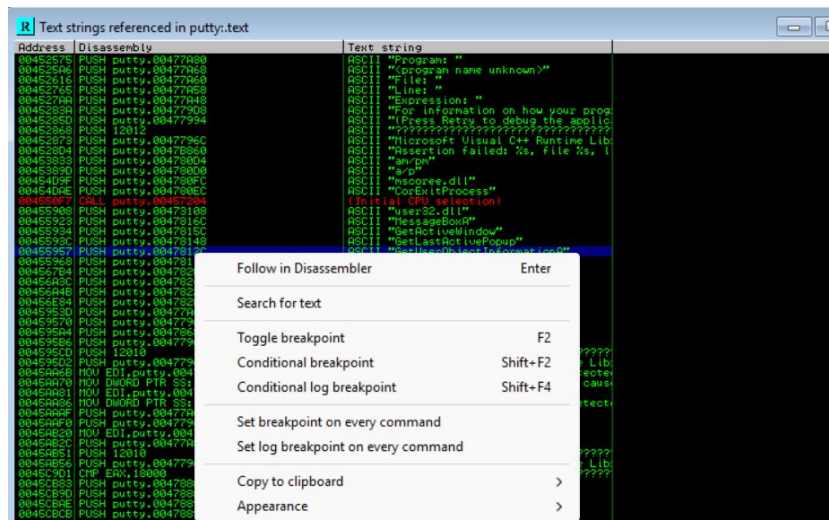
Đây là bước kiểm chứng thành công: cửa sổ Putty xuất hiện cho thấy chương trình vẫn chạy, trong khi việc cổng 4444 đang ở trạng thái LISTENING chứng tỏ mã độc đã cài cắm thành công và đang sẵn sàng chờ kết nối điều khiển từ xa.



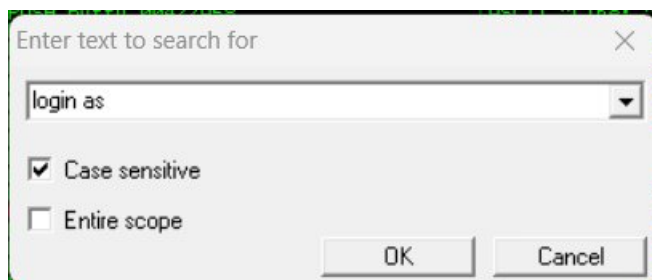
Đây là tính năng tìm kiếm của Immunity Debugger. Bạn dùng nó để định vị các đoạn mã, hàm hoặc chuỗi văn bản cụ thể trong putty.exe, giúp tìm ra chính xác địa chỉ bộ nhớ cần can thiệp để chèn lệnh chuyển hướng (JMP)



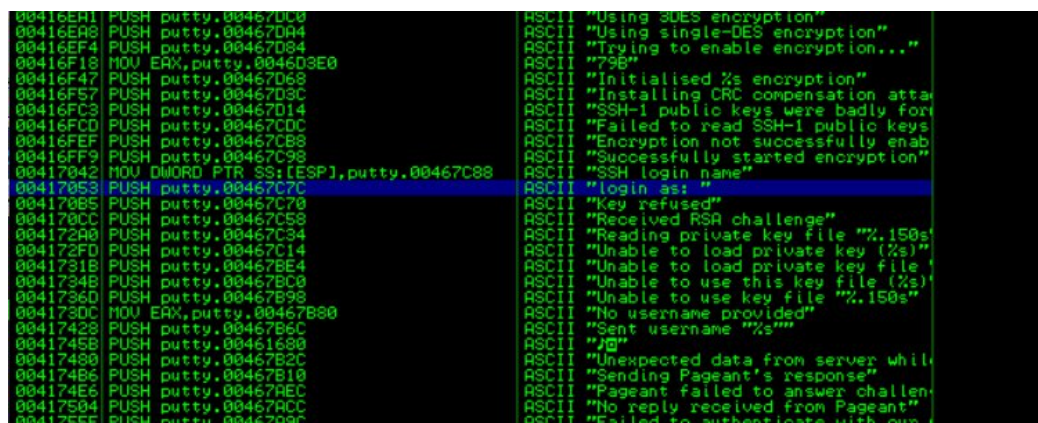
Danh sách các địa chỉ tham chiếu chuỗi trong file putty.text hiển thị vị trí chứa các chuỗi ký tự như tên hàm, thông báo hay đường dẫn. Đây là cửa sổ tham khảo bình thường của Immunity Debugger, không liên quan đến shellcode hay backdoor.



Hộp thoại tìm kiếm chuỗi "login as" trong Immunity Debugger, dùng để xác định vị trí dòng chữ yêu cầu đăng nhập trong chương trình.



Các lệnh PUSH 0x04E704C lặp lại tại nhiều địa chỉ trong vùng nhớ của file putty.exe là kết quả của việc Immunity Debugger giải mã dữ liệu nhị phân thành mã assembly. Vùng nhớ này không chứa mã thực thi hợp lệ mà là dữ liệu thô (raw data) hoặc bảng mã hóa, không liên quan đến shellcode bạn đã chèn vào section .NewSec ở địa chỉ 00484000



Danh sách này chứa toàn bộ các đoạn văn bản (text strings) nằm trong putty.exe.

Mục đích: Tìm dòng chữ "login as:". Khi tìm thấy, bạn sẽ xác định được địa chỉ bộ nhớ (0041CB6E) ngay phía trên nó. Đó chính là điểm neo để bạn chèn lệnh JMP, ép chương trình chạy sang đoạn mã độc trước khi hiển thị khung đăng nhập.

```
0041C703 PUSH putty.00461680 ASCII "Server rejected new password"
0041C733 PUSH putty.00468058 ASCII "New SSH password"
0041C75A PUSH putty.0045D6EC ASCII "%.s"
0041C771 PUSH putty.00468D1C ASCII "Current password (blank for prev"
0041C78C PUSH putty.00468D04 ASCII "Enter new password: "
0041C7A4 PUSH putty.00468CEC ASCII "Confirm new password: "
0041C84F PUSH putty.00468CD0 ASCII "Passwords do not match!\n"
0041C8A1 PUSH putty.00467604 ASCII "Unable to authenticate"
0041C8D7 MOV DWORD PTR SS:[ESP],putty.004691F0 ASCII "ssh-connection"
0041C8E9 MOV DWORD PTR SS:[ESP],putty.00467394 ASCII "password"
0041C93B PUSH putty.00468CBC ASCII "Sent new password"
0041CA11 PUSH putty.00468CAC ASCII "Access granted"
0041CABA PUSH putty.00466488 ASCII "..\ssh.c"
0041CABF PUSH putty.00468C8C ASCII "s->type == AUTH_TYPE_PASSWORD"
0041CACF PUSH putty.00468C6C ASCII "Password authentication failed"
0041CADF PUSH putty.0046786C ASCII "Access denied!\n"
0041CB5C MOV DWORD PTR SS:[ESP],putty.00467C88 ASCII "SSH login name"
0041CB6E PUSH putty.00467C7C ASCII "login as: "
0041CBCC PUSH putty.00468C54 ASCII "Using username \"%s\".\n"
```

Đây là vùng dữ liệu của chương trình, chứa các địa chỉ và giá trị mà PuTTY cần dùng khi chạy. Các lệnh PUSH và MOV hiển thị ở đây không phải là mã thực thi mà chỉ là dữ liệu thô, nên việc đặt breakpoint tại đây là không cần thiết và không ảnh hưởng đến shellcode hay quá trình backdoor.

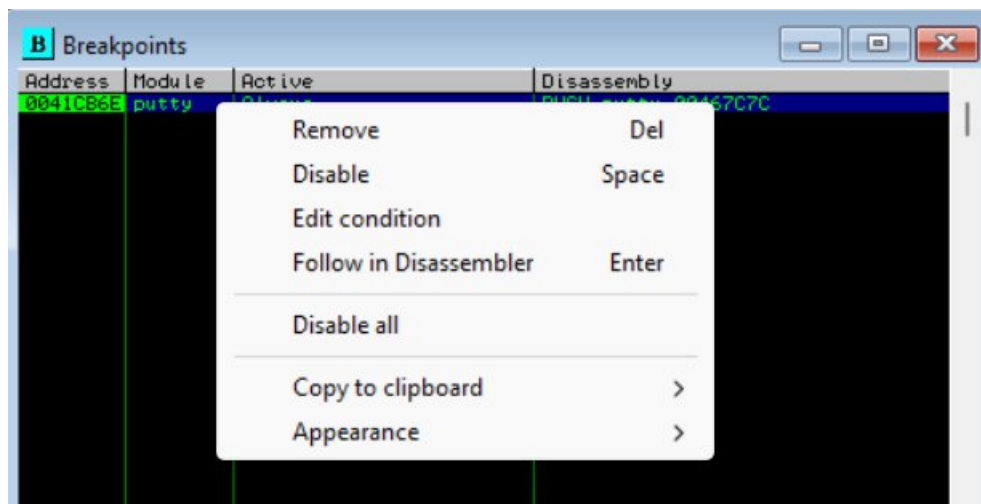
```
0041C6E8 MOV EDI,putty.00468D6C ASCII "Server rejected new password"
0041C703 PUSH putty.00461680 ASCII "New SSH password"
0041C733 PUSH putty.00468058 ASCII "%.s"
0041C75A PUSH putty.0045D6EC ASCII "Current password (blank for prev"
0041C771 PUSH putty.00468D1C ASCII "Enter new password: "
0041C78C PUSH putty.00468D04 ASCII "Confirm new password: "
0041C7A4 PUSH putty.00468CEC ASCII "Passwords do not match!\n"
0041C84F PUSH putty.00468CD0 ASCII "Unable to authenticate"
0041C8A1 PUSH putty.00467604 ASCII "ssh-connection"
0041C8D7 MOV DWORD PTR SS:[ESP],putty.004691F0 ASCII "password"
0041C8E9 MOV DWORD PTR SS:[ESP],putty.00467394 ASCII "Sent new password"
0041C93B PUSH putty.00468CBC ASCII "Access granted"
0041CA11 PUSH putty.00468CAC ASCII "..\ssh.c"
0041CABF PUSH putty.00468C8C ASCII "s->type == AUTH_TYPE_PASSWORD"
0041CACF PUSH putty.00468C6C ASCII "Password authentication failed"
0041CADF PUSH putty.0046786C ASCII "Access denied!\n"
0041CB5C MOV DWORD PTR SS:[ESP],putty.00467C88 ASCII "SSH login name"
0041CB6E PUSH putty.00467C7C ASCII "login as: "
0041CBCC PUSH putty.00468C54 ASCII "Using username \"%s\".\n"
```

Cửa sổ PuTTY Configuration hiển thị khi bạn chạy file putty-DuyKhanh.exe. Việc cửa sổ này xuất hiện chứng tỏ file vẫn chạy và hiển thị giao diện bình thường như một chương trình PuTTY hợp pháp, giúp che giấu hành vi backdoor.

Task 2: Alter the Login Message

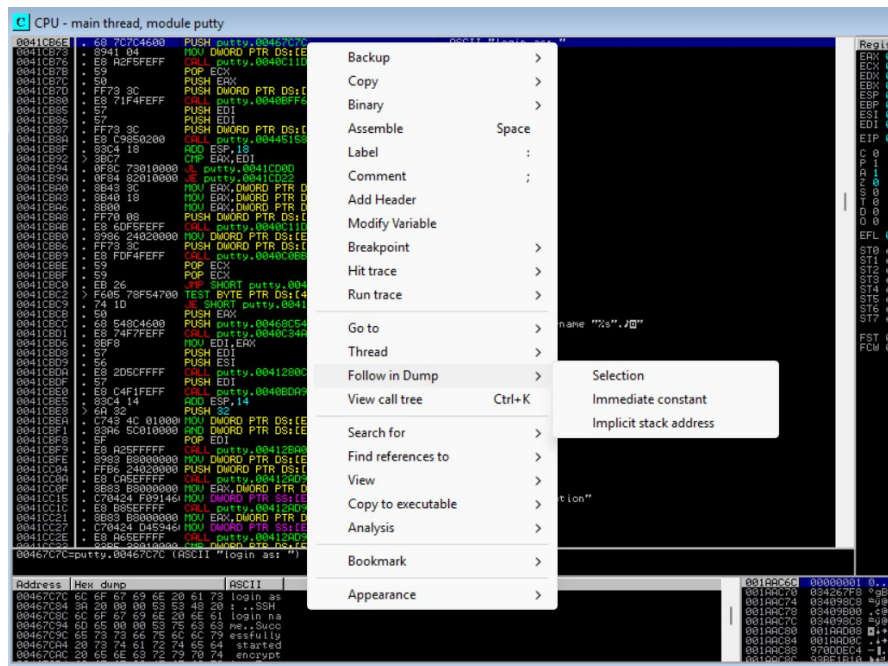
Removing the Breakpoint

Cửa sổ Breakpoints hiển thị danh sách các điểm dừng đã đặt, với địa chỉ và lệnh tương ứng. Tại đây, có thể xóa (Remove), tắt (Disable), hoặc nhảy đến vị trí đặt breakpoint (Follow in Disassembler). Sau khi chụp ảnh xong, vào đây xóa breakpoint để tiếp tục sửa chữ "login as".



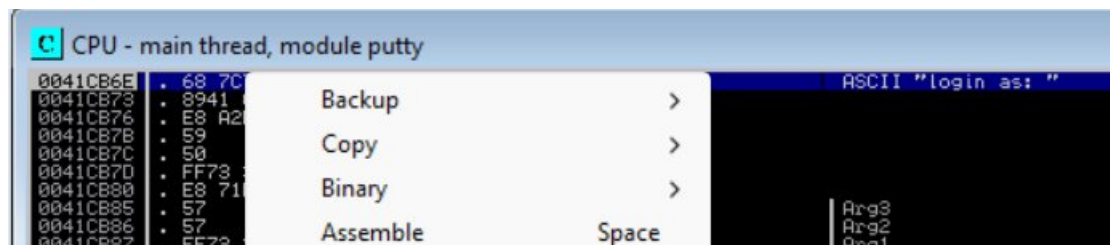
Viewing the Stored Message

Cửa sổ CPU là nơi hiển thị mã Assembly của chương trình, giúp xem lệnh, đặt breakpoint tại địa chỉ 0041CB6E và sửa lệnh. Cửa sổ Breakpoints liệt kê các điểm dừng đã đặt, cho phép xóa hoặc tắt chúng sau khi chụp ảnh. Cửa sổ Hex Dump hiển thị dữ liệu thô trong bộ nhớ, dùng để xem và sửa trực tiếp chuỗi "login as" thành tên của mình



Skipping the First Letter In the Message

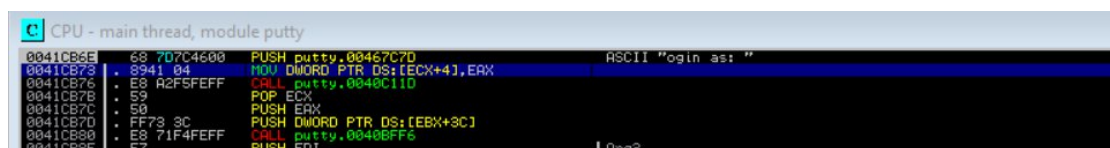
Cửa sổ CPU hiển thị mã Assembly để đặt breakpoint và sửa lệnh. Cửa sổ Hex Dump dùng để sửa trực tiếp chuỗi "login as" trong bộ nhớ. Cửa sổ Breakpoints quản lý các điểm dừng đã đặt. Cửa sổ Text Strings Referenced giúp tìm nhanh vị trí của "login as"



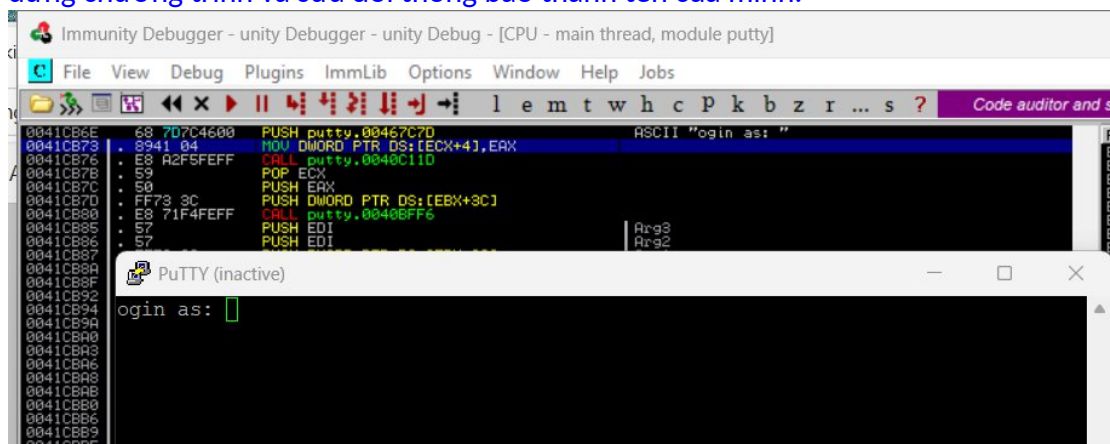
Cửa sổ "Assemble at 0041CB6E" dùng để sửa đổi lệnh tại địa chỉ đã chọn. Trong lab, tại đây đang hiển thị lệnh PUSH 467C7D, có chức năng đưa địa chỉ của chuỗi "login as" vào ngăn xếp để chương trình hiển thị. Khi bỏ chọn ô "Fill with NOP's" và bấm Assemble, lệnh mới sẽ được ghi đè.



Cửa sổ CPU hiển thị mã Assembly của chương trình. Tại địa chỉ 0041CB6E có lệnh PUSH đẩy địa chỉ chuỗi "login as:" vào ngăn xếp, bên cạnh hiển thị dòng ASCII. Các lệnh phía sau gồm MOV, PUSH, CALL... thực hiện việc xử lý và hiển thị chuỗi này.

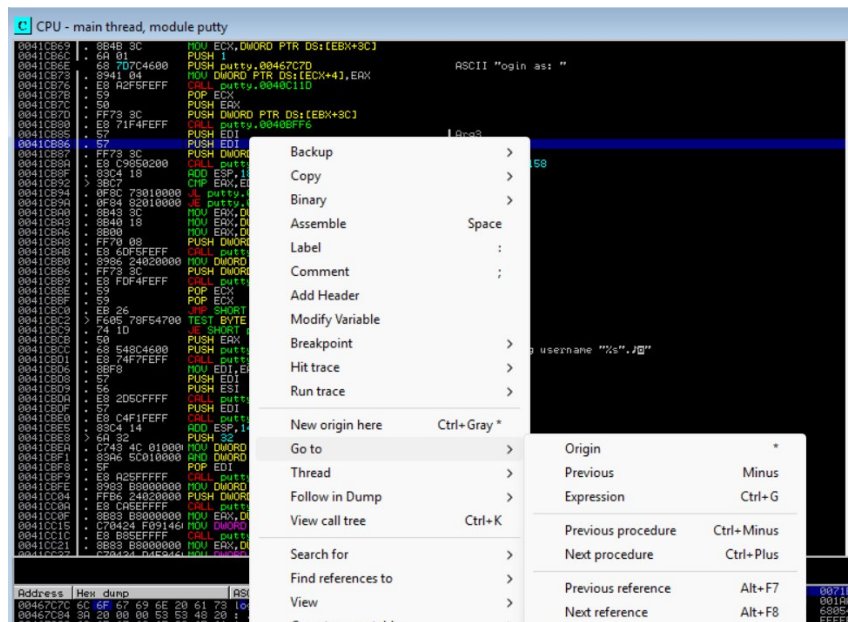


Cửa sổ CPU hiển thị mã Assembly của chương trình. Tại địa chỉ 0041CB6E có lệnh PUSH đưa chuỗi "login as:" vào ngăn xếp để hiển thị. Đây là vị trí đặt breakpoint để dừng chương trình và sửa đổi thông báo thành tên của mình.

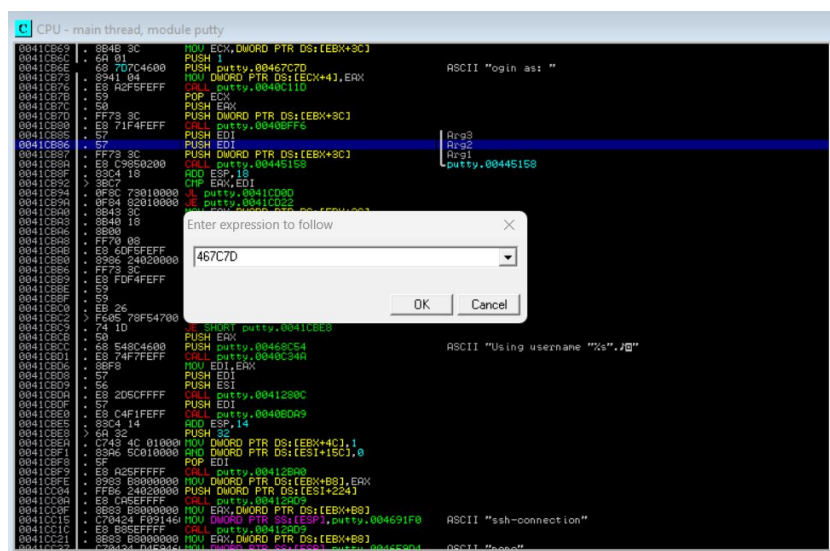


Inserting Your Name

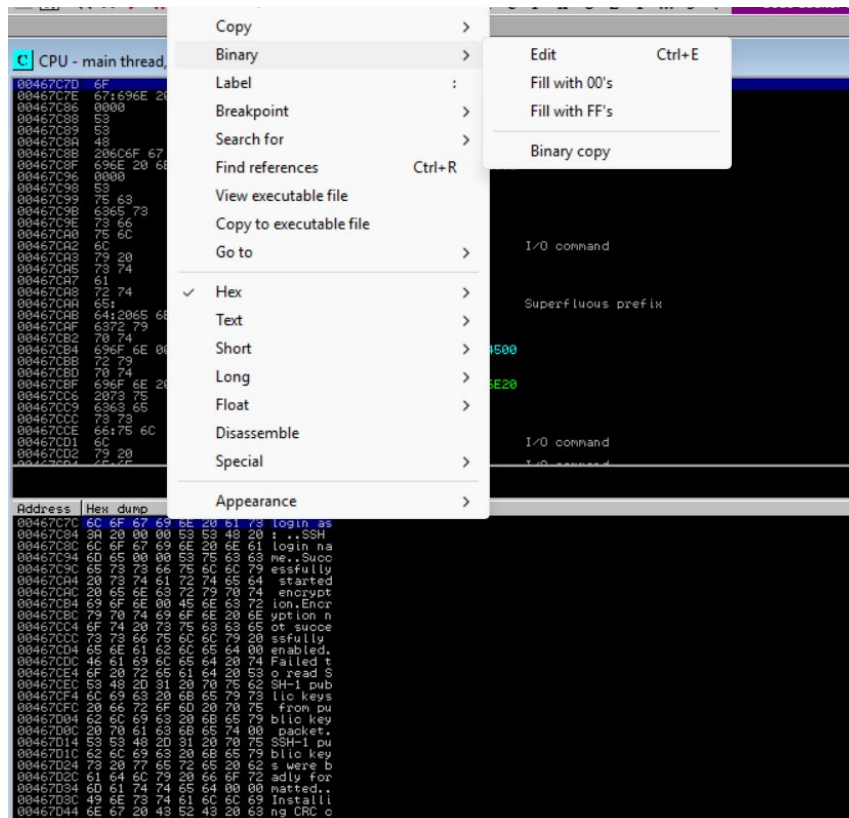
Giao diện Hex Dump cho phép bạn quan sát trực tiếp các byte dữ liệu thô của chương trình dưới dạng mã Hex. Trong bài Lab, bạn sử dụng cửa sổ này để xác nhận rằng các lệnh mã máy đã được ghi đè thành công; cụ thể, bạn có thể kiểm tra xem lệnh JMP đã được chèn vào đúng vị trí để "lái" chương trình chạy vào đoạn mã độc hay chưa trước khi tiến hành lưu lại tệp tin đã bị sửa đổi.



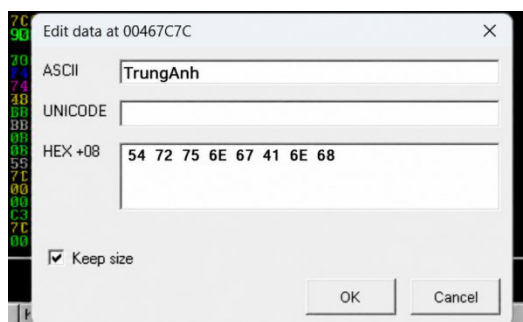
Cửa sổ này dùng để nhảy đến địa chỉ bộ nhớ 467C7D - nơi chứa chuỗi "login as" trong Immunity Debugger, nhằm xem và sửa đổi nội dung của chuỗi đó trong cửa sổ Hex Dump.



Cửa sổ này hiển thị nội dung bộ nhớ tại địa chỉ 467C7D, nơi chứa chuỗi "login as". Các giá trị hex hiển thị là mã ASCII của từng ký tự. Đây là vùng nhớ cần sửa để thay đổi thông báo thành tên mong muốn.



Cửa hộp thoại "Edit data at 00467C7C" dùng để sửa trực tiếp dữ liệu tại vùng nhớ chứa chuỗi "login as". Trong lab, đã nhập tên "TrungAnh" vào ô ASCII, tương ứng với mã hex "54 72 75 6E 67 41 6E 68" và chọn "Keep size" để giữ nguyên độ dài 8 byte, sau đó bấm OK.



Saving the Modified EXE

Cửa sổ Hex Dump hiển thị nội dung bộ nhớ sau khi đã sửa chuỗi "login as" thành tên "TrungAnh". Tại địa chỉ 00467C7C, dữ liệu hex "54 72 75 6E 67 41 6E 68" tương ứng với ASCII "TrungAnh". Các dòng bên dưới hiển thị phần còn lại của chuỗi gốc và các chuỗi khác trong chương trình.

Address	Hex dump	ASCII	
00467C7C	44 75 79 4B 68 61 6E 68	DuyKhanh	
00467C84	3A 20 00 00 53 53 48 20	...SSH	
00467C8C	6C 6F 67 69 6E 20 6E 61	login na	
00467C94	6D 65 00 00 53 75 63 63	me..Successfully	
00467C9C	65 73 73 66 75 6C 6C 79	starteld	
00467CA4	20 73 74 61 72 74 65 64	started	
00467CAC	20 65 6E 63 72 79 70 74	encrypt	
00467CB4	69 6F 6E 00 45 6E 63 72	ion.Encr	
00467CBC	79 70 74 69 6F 6E 20 6E	ryption n	
00467CC4	6F 74 20 73 75 63 63 65	ot succe	
00467CCC	73 73 66 75 6C 6C 79 20	ssfully	
00467CD4	65 6E 61 62 6C 65 64 00	enabled.	
00467CDC	46 61 69 6C 65 64 20 74	Failed t	
00467CE4	6F 20 72 65 61 64 20 53	o read S	
00467CEC	53 48 20 31 20 70 75 62	SH-1 pub	

Copto executable file

started >

✓ Hex >

Text >

Short >

Long >

Cửa sổ "Save file" xuất hiện sau khi chọn "Copy to executable file" từ menu click phải trong Hex Dump, dùng để lưu file EXE đã sửa đổi. Ảnh hiển thị vùng bộ nhớ chứa chuỗi "TrungAnh" đã thay thế "login as" cùng các dữ liệu xung quanh. Các tùy chọn Hex, Text, Short... bên phải cho phép thay đổi định dạng hiển thị. Cần đặt tên mới như "putty-TrungAnh.exe" rồi nhấn Save.

Address	Hex dump	ASCII	
00067C7C	44 75 79 4B 68 61 6E 68	DuyKhanh	
00067C84	3A 20 00 00 53 53 48 20	:..SSH	
00067C8C	6C 6F 67 69 6E 20 6E 61	login na	
00067C94	6D 65 00 00 53 75 63 63	loginSuco	
00067C9C	65 73 73 66 75 6C 6C 79	essfully	
00067CA4	20 73 74 61 72 74 65 64	started	
00067CAC	20 65 6E 63 72 79 70 74	encrypt	
00067CB4	69 6F 6E 00 45 6E 63 73	ion.Encr	
00067CBC	79 70 74 69 6F 6E 20 6E	ryption n	
00067CC4	6F 74 20 73 75 63 63 65	ot succe	
00067CCC	73 73 66 75 6C 6C 79 20	ssfully	
00067CD4	65 6E 61 62 6C 65 64 00	snabled.	
00067CDC	46 61 69 6C 65 64 20 74	Failed t	
00067CE4	6F 20 72 65 61 64 20 53	o read S	
00067CEC	53 48 20 31 20 70 75 62	SH-1 pub	
00067CF4	6C 69 63 20 60 65 79 73	lic keys	
00067CFC	20 66 72 6F 20 20 70 75	from pu	
00067D04	62 6C 69 63 20 68 65 79	blic key	
00067D0C	20 70 61 62 68 6F 74 00	public	

Save file

Go to offset Ctrl+G

View image in CPU Dump

✓ Hex >

Text >

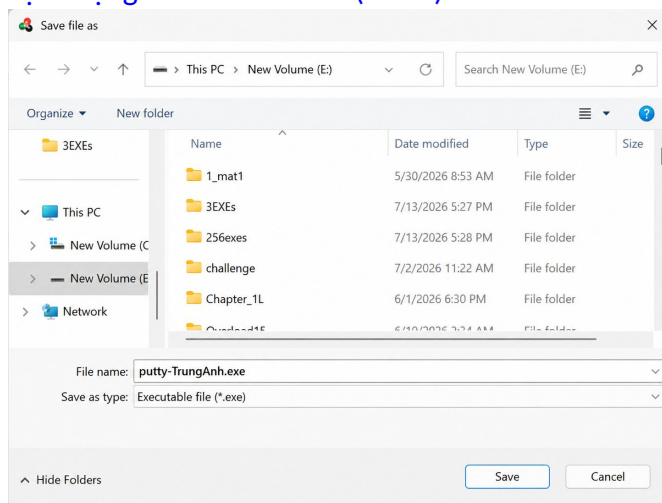
Short >

Long >

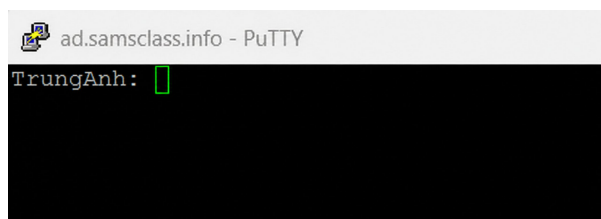
Float >

Disassemble

Cửa sổ "Save file as" là hộp thoại lưu file, xuất hiện sau khi đã sửa dữ liệu trong Hex Dump và chọn "Copy to executable file". Tại đây, cần đặt tên file mới là "putty-TrungAnh.exe" (thay "DuyKhanh" bằng tên của mình), chọn thư mục lưu, giữ nguyên định dạng "Executable file (*.exe)" rồi bấm Save.



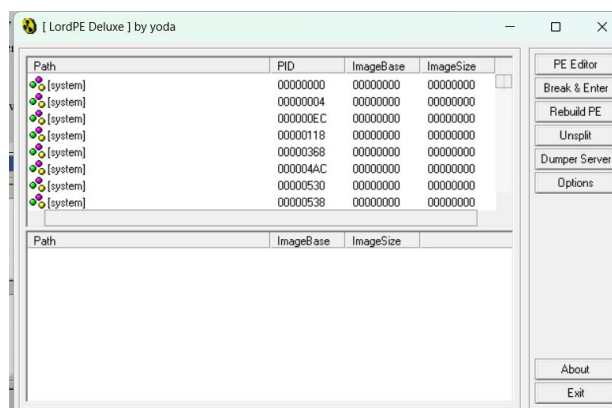
Cửa sổ này là kết quả cuối cùng của bài lab, hiển thị chương trình Putty đã được sửa đổi thành công. Sau khi thực hiện các bước sửa chuỗi "login as" thành tên "TrungAnh" và lưu file EXE mới, khi chạy file đã sửa và kết nối tới địa chỉ ad.samsclass.info, thông báo đăng nhập đã thay đổi thành "TrungAnh:" thay vì "login as:" như ban đầu.



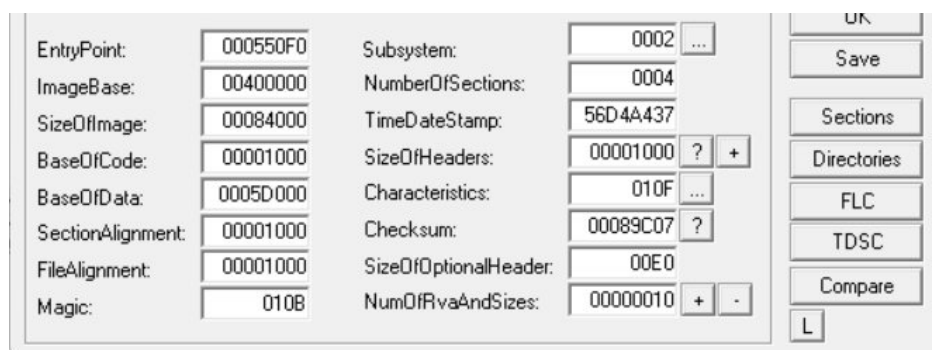
Proj 8b: EXE With Trojan Code in a New Section

Task 1: Add a Section with LordPE

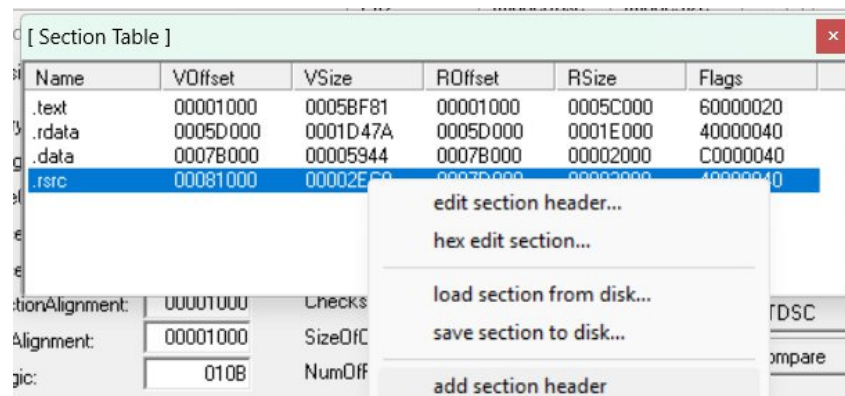
Đây là giao diện chính của LordPE, công cụ dùng để chỉnh sửa cấu trúc file PE (Portable Executable). Trong lab này, LordPE được sử dụng để thêm một section mới (.NewSec) vào file putty.exe, tạo không gian trống để chèn shellcode. Các chức năng như "PE Editor" dùng để mở và sửa file, "Sections" dùng để thêm section mới, và "Save" để lưu thay đổi.



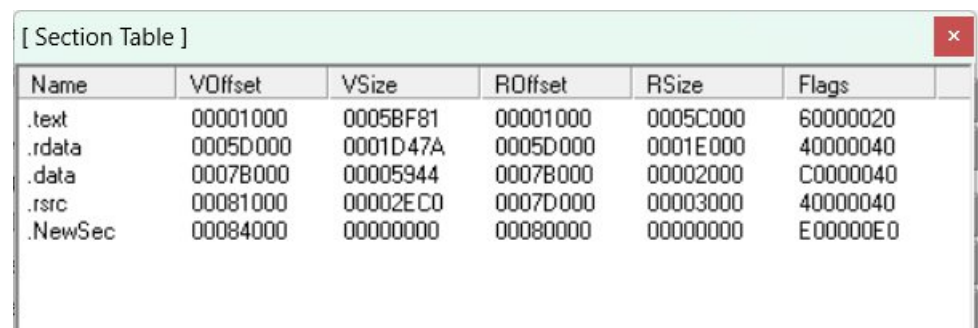
Cửa sổ PE Editor của LordPE hiển thị thông tin cấu trúc file putty-newsec-trunganh.exe bao gồm EntryPoint (điểm vào), ImageBase (địa chỉ cơ sở), SizeOfImage (kích thước file trong bộ nhớ) và NumberOfSections (số section hiện có).



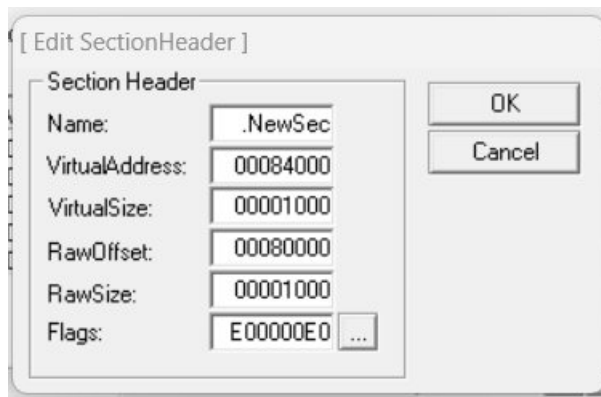
Bảng Section Table trong LordPE hiển thị các section hiện có của file putty.exe: .text (chứa mã lệnh), .rdata (dữ liệu chỉ đọc), .data (dữ liệu có thể ghi), .rsrc (tài nguyên). Trong lab này, bạn click chuột phải và chọn "add section header" để tạo section .NewSec mới với kích thước 1000, làm nơi chứa shellcode, sau đó nhấn Save để lưu thay đổi.



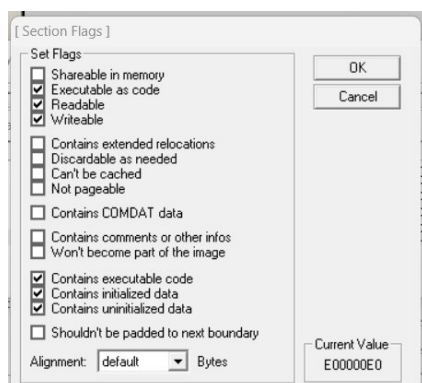
Bảng Section Table hiển thị section .NewSec đã được thêm thành công với VOffset 00084000 và ROffset 00080000. Section này có kích thước 1000 và sẽ chứa shellcode. Sau khi thêm, bạn nhấn Save để lưu file và tiếp tục dùng Immunity Debugger để chuyển hướng luồng thực thi đến địa chỉ 00484000.



Cửa sổ PE Editor hiển thị thông tin tổng quan về file. Cửa sổ Section Table liệt kê các section, tại đây bạn thêm .NewSec. Cửa sổ Edit SectionHeader xác nhận thông tin section mới với VirtualAddress 00084000, dùng làm địa chỉ cho lệnh JMP trong Immunity Debugger.



Cửa sổ cấu hình quyền cho section `.NewSec`. Các tùy chọn đã chọn bao gồm: Executable as code, Readable, Writeable, Discardable as needed, Can't be cached, Not pageable, Won't become part of the image, Contains executable code, Contains initialized data, Contains uninitialized data. Current Value là E00000E0, đại diện cho tổ hợp quyền này



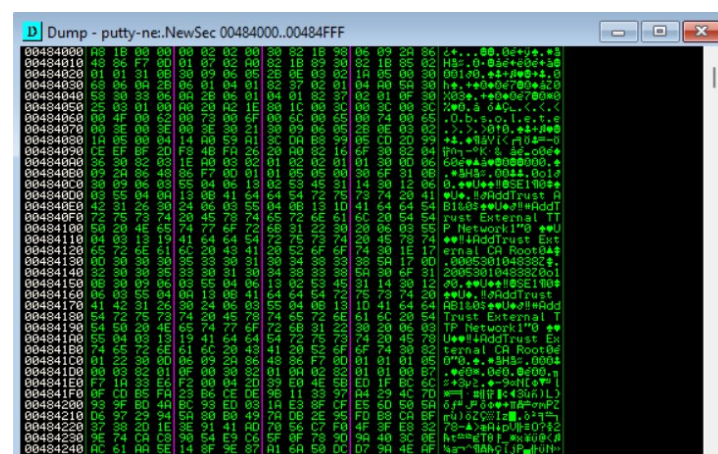
Task 2: Redirecting Code Execution with Immunity

Using Immunity to Examine the NewSec Section

Cửa sổ Memory Map trong Immunity Debugger, hiển thị các vùng nhớ của tiến trình. Nó liệt kê địa chỉ, kích thước, owner và section tương ứng. Vùng nhớ này không liên quan đến shellcode hay backdoor, chỉ là thông tin bình thường về bố cục bộ nhớ của chương trình

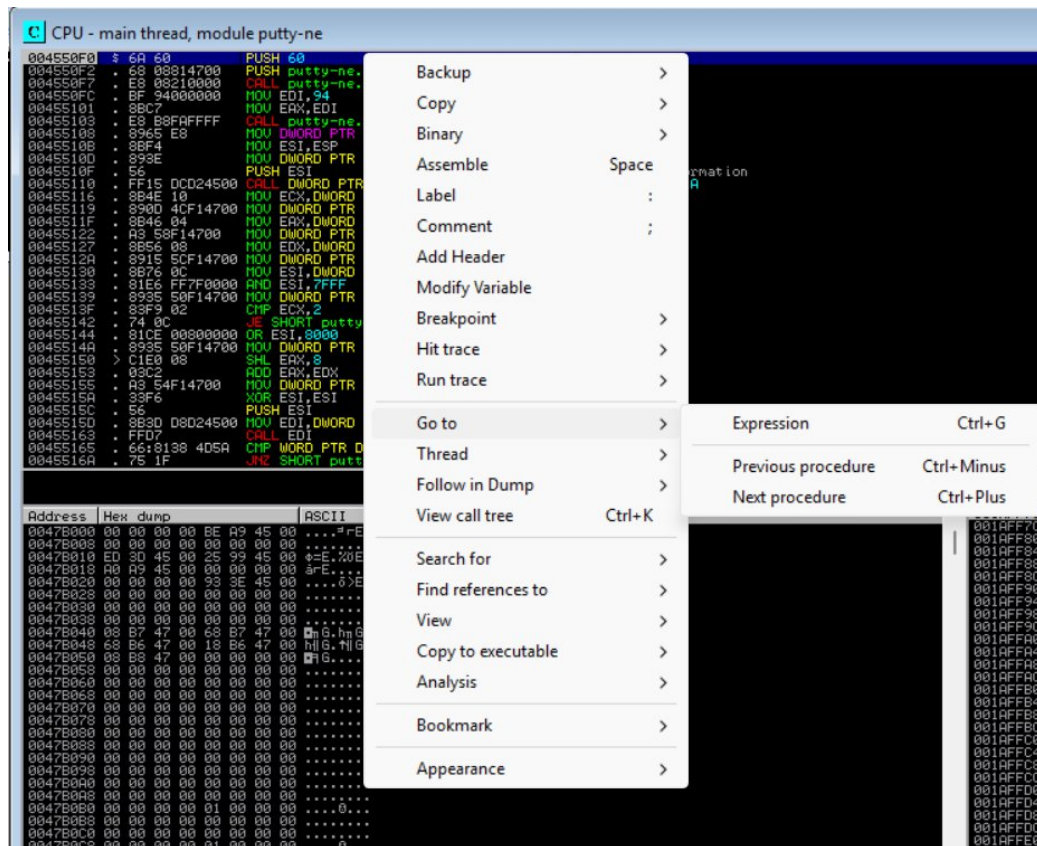
Address	Size	Owner	Section	Contains
00010000	00011000			
00030000	00001000			
00040000	0002A000			
000A5000	0000B000			
001A5000	00002000			
001A0000	00003000			
001B0000	00004000			stack of na
001C0000	00002000			
001D0000	00002000			
001E0000	00011000			
0020A000	00004000			
0020E000	00004000			data block
00212000	00004000			data block
00216000	00004000			data block
0021A000	00002000			data block
00400000	00001000	putty-ne		FE header
00401000	0005C000	putty-ne	.text	code
0045D000	0001E000	putty-ne	.rdata	imports
00475000	00006000	putty-ne	.data	data
00481000	00003000	putty-ne	.rsrc	resources
00484000	00001000	putty-ne	.NewSec	
00485000	00011000			

Cửa sổ Dump hiển thị dữ liệu thô tại section .NewSec (địa chỉ 00484000). Trước khi chèn shellcode, vùng này chứa các giá trị mặc định (00) và một số dữ liệu rác, không phải mã lệnh. Đây là vùng trống bạn sẽ dùng để chèn shellcode bằng HxD hoặc Immunity



Using Immunity to Redirect Code Execution

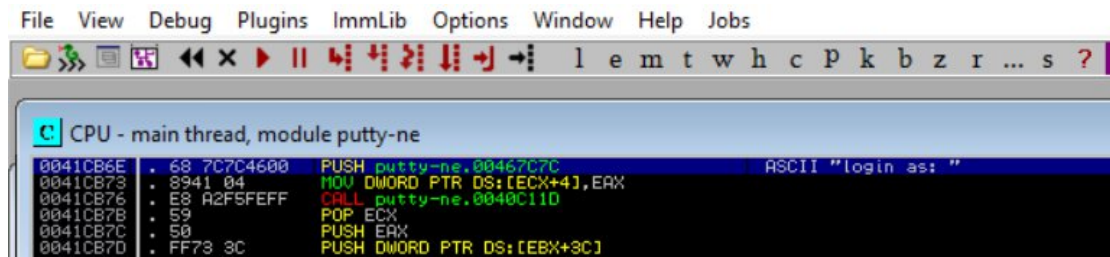
Menu điều hướng (Go to) trong Immunity Debugger, cho phép bạn nhảy nhanh đến một địa chỉ bộ nhớ hoặc biểu thức (expression) cụ thể. Trong bài Lab, tính năng này giúp bạn di chuyển tức thời tới các vị trí quan trọng đã xác định trước đó, như điểm bắt đầu của mã lệnh hoặc phần Section mới vừa tạo, giúp việc chỉnh sửa và chèn lệnh JMP trở nên chính xác và nhanh chóng hơn.



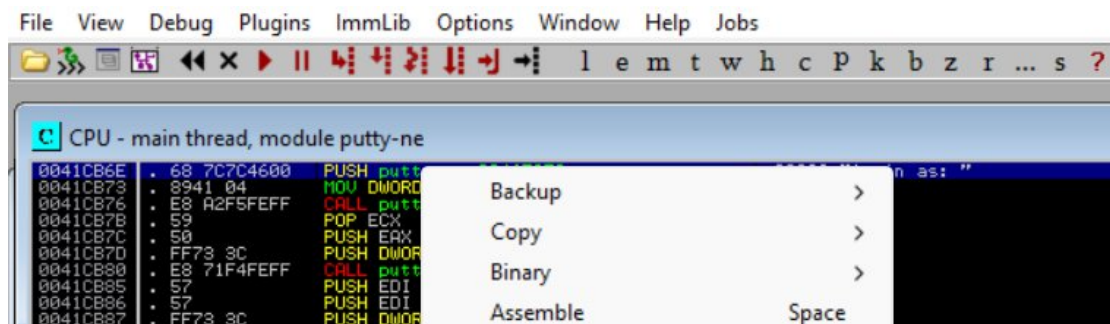
Đây là hộp thoại "Go to Expression" trong Immunity Debugger, nhập địa chỉ 41CB6E để di chuyển đến vị trí chứa lệnh in chuỗi "login as:". Đây là địa chỉ bạn cần sửa thành lệnh JMP 00484000 để chuyển hướng luồng thực thi sang shellcode trong section .NewSec



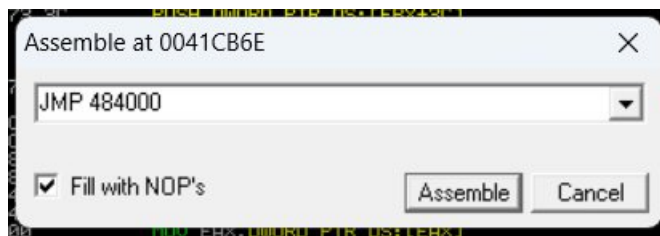
Đây là vị trí chứa lệnh in chuỗi "login as: " trong chương trình. Bạn sẽ sửa lệnh PUSH tại địa chỉ 0041CB6E thành JMP 00484000 để chuyển hướng luồng thực thi sang shellcode. Các lệnh phía sau (MOV, PUSH,...) là mã bình thường của chương trình, không liên quan đến backdoor.



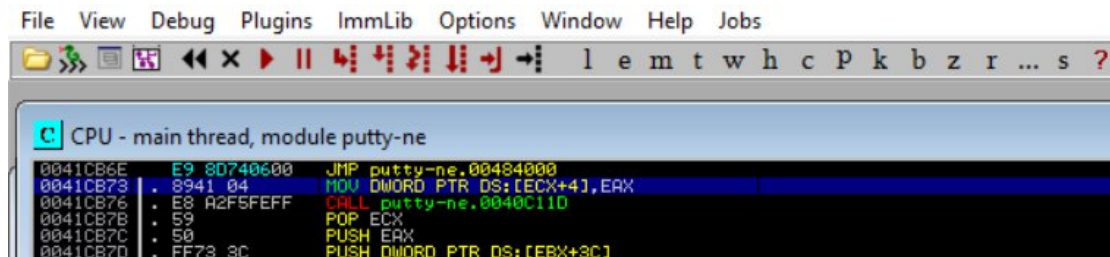
Trong ảnh này, tại địa chỉ 0041CB6E, dòng lệnh vẫn hiển thị là PUSH 0x7C7C4600, tức là chương trình vẫn đang thực hiện thao tác đẩy giá trị lên stack để chuẩn bị in dòng chữ "login as:". Đây là hành vi bình thường của file putty.exe gốc



Tại địa chỉ 0041CB6E, bạn nhập lệnh JMP 484000 và chọn tùy chọn "Fill with NOP's" để chèn các lệnh NOP (No Operation) vào khoảng trống còn lại. Thao tác này sẽ ghi đè lên các byte không cần thiết và đảm bảo luồng thực thi nhảy chính xác đến shellcode tại 00484000. Sau khi click Assemble, lệnh gốc PUSH sẽ bị thay thế hoàn toàn.



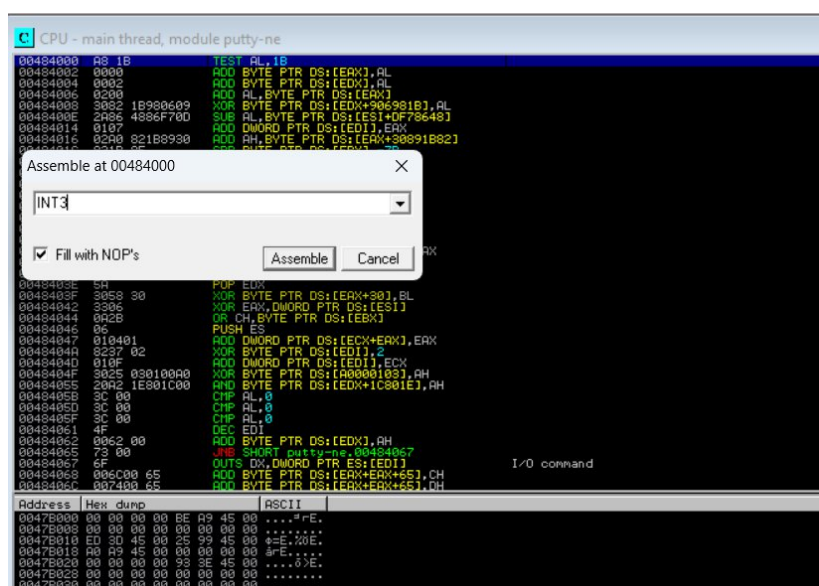
Tại địa chỉ 0041CB6E, lệnh đã được sửa thành `JMP putty-ne.00484000`, chuyển hướng luồng thực thi sang shellcode trong section `.NewSec`.



```
File View Debug Plugins Immlib Options Window Help Jobs
l e m t w h c P k b z r ... s ?
CPU - main thread, module putty-ne
0041CB6E E9 8D740600 JMP putty-ne.00484000
0041CB73 8941 04 MOV DWORD PTR DS:[ECX+41],EAX
0041CB76 E8 A2F5FEFF CALL putty-ne.0040C11D
0041CB7B 59 POP ECX
0041CB7C 50 PUSH EAX
0041CB7D FF73 3C PUSH DWORD PTR DS:[EBX+3C]
```

Adding Trojan Code

Tại địa chỉ 00484000, bạn đang xem vùng nhớ của section `.NewSec` sau khi đã chèn shellcode. Các lệnh hiển thị như `LES R8, 10` và `R9 10` là do Immunity Debugger giải mã dữ liệu shellcode thành mã assembly, nhưng thực chất đây là các byte shellcode thô. Sau khi chạy, shellcode sẽ mở cổng 4444 và chờ kết nối.



```
CPU - main thread, module putty-ne
00484000 A9 1B TEST AL,1B
00484002 0000 ADD BYTE PTR DS:[EAX],AL
00484004 0002 ADD BYTE PTR DS:[EDX],AL
00484006 0200 ADD AL,BYTE PTR DS:[EAX]
00484008 3002 XOR BYTE PTR DS:[EDX+00001B],AL
0048400C 2006 4806700 SUB AL,BYTE PTR DS:[ESI+0F706480]
00484014 0107 ADD DWORD PTR DS:[EDI],EAX
00484016 0200 821B8900 ADD AH,BYTE PTR DS:[EAX+30891B82]
0048401E 0000 ADD AL,BYTE PTR DS:[EAX]
00484020 5A POP EAX
00484022 3008 00 XOR BYTE PTR DS:[EAX+00],BL
00484024 3306 XOR EAX,DWORD PTR DS:[ESI]
00484026 002B OR CH,BYTE PTR DS:[EBX]
00484028 05 PUSH ES
0048402A 010401 ADD DWORD PTR DS:[ECX+EAX],EAX
0048402C 8237 02 XOR BYTE PTR DS:[EDI],2
0048402E 010F ADD DWORD PTR DS:[EDI],ECX
00484030 3005 03010000 XOR EAX,DWORD PTR DS:[00000103],AH
00484032 20A2 1E001C00 AND BYTE PTR DS:[EDX+1C001E],AH
00484034 3C 00 CMP AL,0
00484036 3C 00 CMP AL,0
00484038 3C 00 CMP AL,0
0048403A 4F DEC EDI
0048403C 0002 00 ADD BYTE PTR DS:[EDX],AH
0048403E 73 00 JNB SHORT putty-ne.00484067
00484040 6F OUTS DX,DWORD PTR ES:[EDI] I/O command
00484042 00C0 65 ADD BYTE PTR DS:[EAX+EAX+00C0],CH
00484044 007400 65 ADD BYTE PTR DS:[EAX+EAX+0074],CH
Address Hex dump ASCII
0047B000 00 00 00 00 BE A9 45 00 .....#E.
0047B008 00 00 00 00 00 00 00 00 .....
0047B010 ED 3D 45 00 25 99 45 00 #=E..0E.
0047B018 A0 A5 00 00 00 00 00 00 #E.....
0047B020 00 00 00 00 93 3E 45 00 ....0E.
0047B028 00 00 00 00 00 00 00 00 .....
0047B030 00 00 00 00 00 00 00 00 .....
```

Tại địa chỉ 00484000, shellcode đã được chèn thành công. Lệnh đầu tiên INT3 là dấu hiệu xác nhận shellcode đã ghi đè lên vùng nhớ này. Các byte như 90 NOP, 0000, 0200, 3082... là mã shellcode thực tế

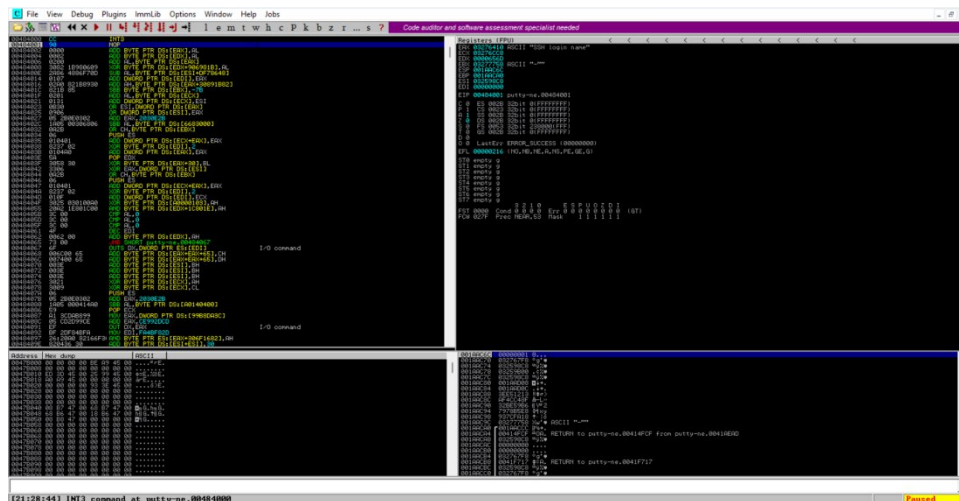
```

CPU - main thread, module putty-ne
00484000 CC      INT3
00484001 90      NOP
00484002 0000    ADD BYTE PTR DS:[EAX],AL
00484004 0002    ADD BYTE PTR DS:[EDX],AL
00484006 0200    ADD AL,BYTE PTR DS:[EAX]
00484008 3082    XOR BYTE PTR DS:[EDX+906981B],AL
0048400E 2A86    SUB AL,BYTE PTR DS:[ESI+0F78648]
00484014 0107    ADD DWORD PTR DS:[EDI],EAX
00484016 02A0    ADD AH,BYTE PTR DS:[EAX+30891B82]
0048401C 821B    SBB BYTE PTR DS:[EBX],-7B
0048401F 0201    ADD AL,BYTE PTR DS:[ECX]
00484021 0131    ADD DWORD PTR DS:[ECX],ESI
00484023 0B30    OR ESI,DWORD PTR DS:[EAX]

```

Running the Modified App in Immunity

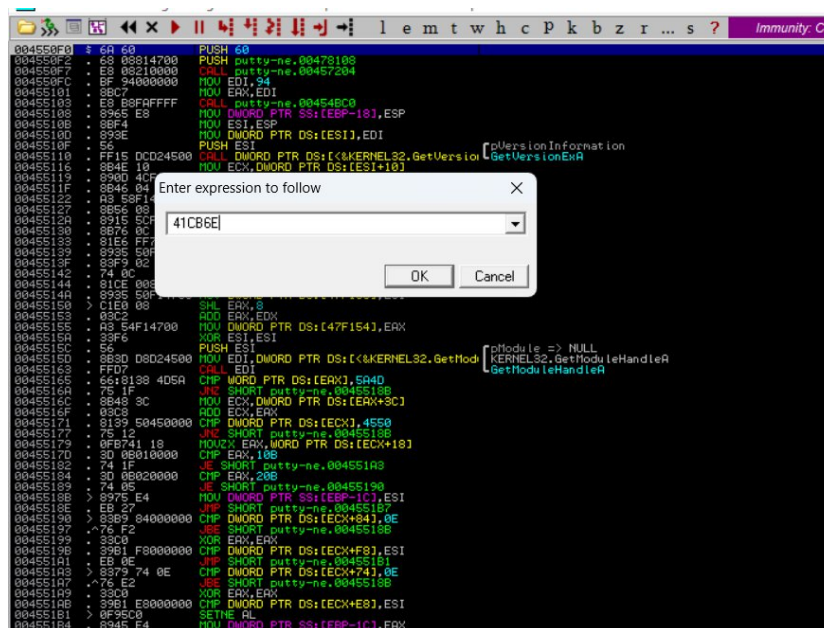
Đây là công cụ Assemble cho phép bạn viết trực tiếp lệnh hợp ngữ để ghi đè mã máy của chương trình. Trong bài Lab, bạn dùng nó để nhập lệnh JMP trở tới đoạn mã độc đã chuẩn bị sẵn, ép chương trình chuyển hướng thực thi ngay khi khởi động nhằm kích hoạt backdoor.



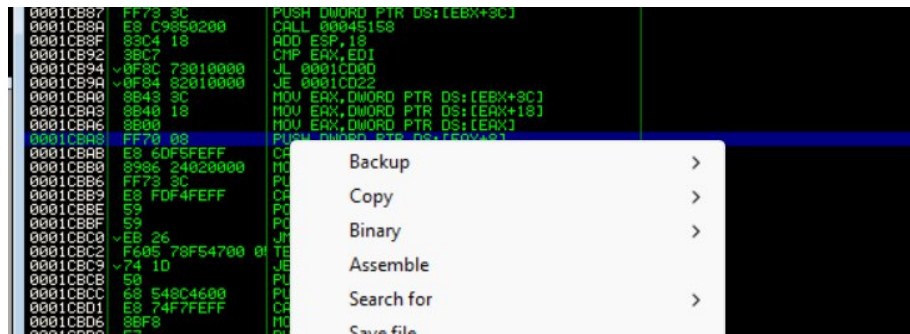
Task 3: Inserting Real Shellcode

Saving the Modified EXE

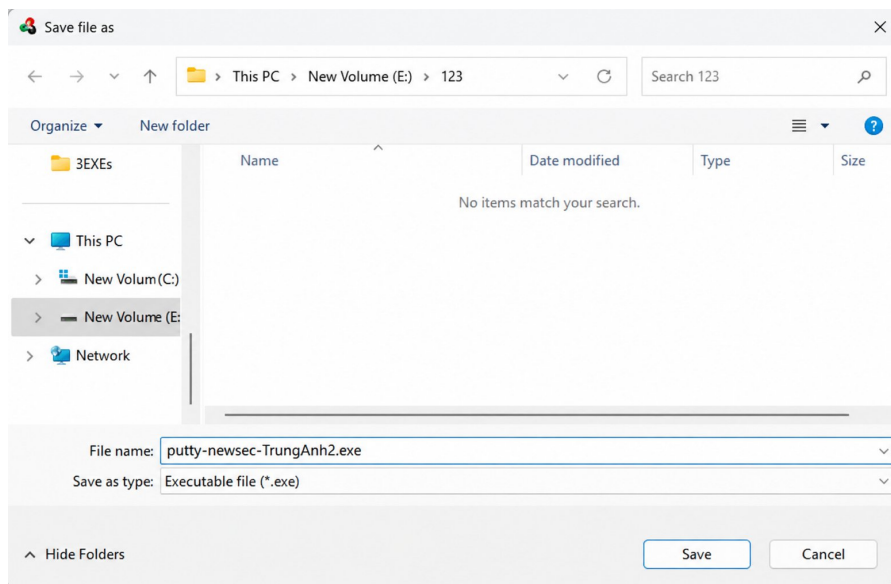
Hộp thoại nhập địa chỉ 41CB6E để di chuyển đến vùng nhớ đó. Tuy nhiên, địa chỉ đúng cần sửa trong lab là 41CB6E (lệnh in "login as:"). Nhập sai sẽ không tìm đúng vị trí cần backdoor



File putty-newsec-trunganh.exe đang được mở trong Immunity Debugger tại vùng nhớ chứa dữ liệu, không phải mã lệnh. Các lệnh ADD ESP, 18 và CALL hiển thị là do Immunity giải mã dữ liệu thô thành assembly, không phải mã thực thi. Vùng này không liên quan đến shellcode hay backdoor

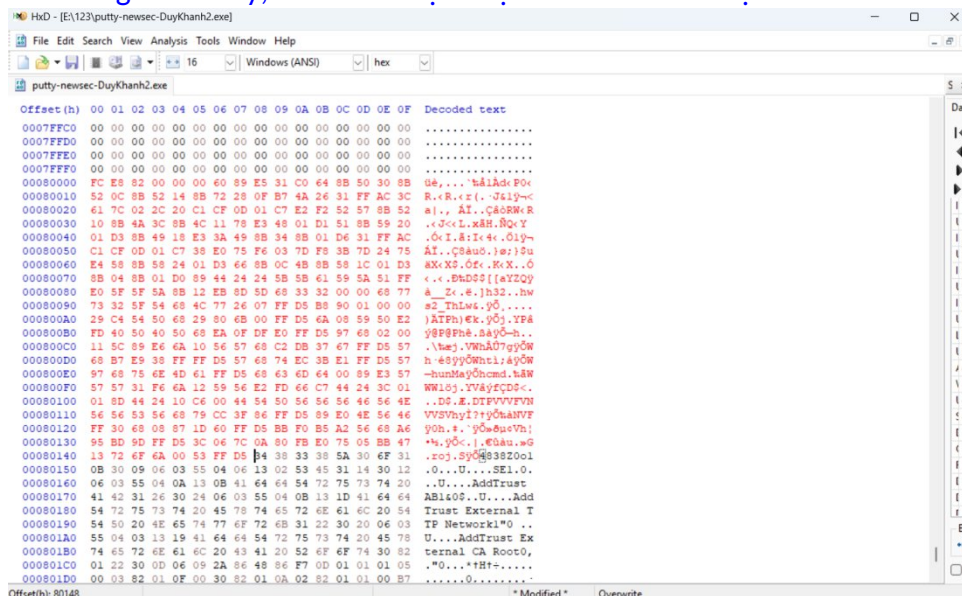


Lưu file đã backdoor với tên putty-newsec-TrungAnh2.exe vào thư mục E:\123\. Đây là bước cuối trong Immunity Debugger để xuất file EXE đã sửa. Sau khi nhấn Save, file sẽ được tạo và sẵn sàng để chạy kiểm tra backdoor. Bạn có thể đóng Immunity và tiến hành chạy file để kiểm tra cổng 4444.




```
File Edit Search View Analysis Tools Window Help
[Icons] 15 Windows (ANSI) hex
putty-newser-DupKhan2.eee

Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded text
0007FFED 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0007FFEE 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0007FFEF 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00080000 48 1B 0D 00 00 02 02 00 82 1B 98 0E 09 2A 86 [...].....0..*..*..*
00080001 48 1B 07 01 07 02 00 A2 1B 98 0E 09 2A 86 HPE.....*..0..*..*
00080002 48 1B 08 30 0A 06 05 2B 0E 03 02 1A 05 00 .....1.....0.....
00080003 68 04 2B 06 01 04 01 B2 37 02 01 0A 5A 30 h.....7.....20
00080004 58 30 04 0A 2B 06 01 04 01 B2 37 02 01 0F X03.....7.....0
00080005 23 03 01 00 A0 21 AE 18 1C 00 3C 3C 00 3C .....8.....c.....c
00080006 00 47 02 62 00 73 00 6F 6C 00 65 00 74 00 65 .....0..b..0..s...e..e
00080007 00 4E 00 3C 00 3C 21 30 09 05 2B 0E 03 02 .....0..0.....0.....
00080008 1A 03 04 14 0A 59 1A 3C DA 88 05 02 D9 88 .....Y.....0..1..1
00080009 CE EF B2 D7 F8 4B 26 20 A0 82 16 6F 82 04 1f -lgi-wkx.....o..0..
0008000A 36 30 82 03 1E A0 03 02 01 02 02 01 01 6F 60 .....0.....0.....
0008000B 09 2A 48 48 16 67 0D 01 01 05 00 3F 31 0B .....*H*P.....0..0..
0008000C 30 09 06 03 55 04 06 13 02 53 74 34 10 12 06 .....0.....SE1.0..
0008000D 03 55 04 13 0B 41 64 54 72 75 74 20 41 .....0.....AddTrust A
0008000E 42 31 30 24 06 03 55 04 0B 13 41 64 54 B105.....U.....Add
0008000F 72 75 74 20 45 78 74 65 72 6E 41 6C 20 54 54 External TTP
00080010 20 4E 45 74 77 67 62 81 22 20 06 03 55 55 TP Network*VQ...U...
00080011 04 03 19 41 64 54 72 75 74 20 45 78 74 20 45 U...AddTrust Ex
00080012 65 72 6E 41 6C 20 43 41 52 5F 6F 74 30 1E 17 External CA Root0...
00080013 00 30 30 33 31 30 31 30 34 17 00 0050301049300...
00080014 32 30 35 33 31 30 34 17 00 00050301049300...
00080015 0B 30 06 03 55 04 06 13 02 53 53 74 34 10 12 .....0.....SE1.0..
00080016 06 03 55 04 13 0B 41 64 54 72 75 74 20 41 .....0.....AddTrust
00080017 41 42 26 30 24 06 03 55 04 0B 13 41 64 54 B105.....U.....Add
00080018 54 72 75 74 20 45 78 74 65 72 6E 41 6C 20 54 54 External TTP
00080019 54 20 4E 45 74 77 67 62 81 22 20 06 03 55 55 TP Network*VQ...U...
0008001A 04 03 19 41 64 54 72 75 74 20 45 78 74 20 45 U...AddTrust Ex
0008001B 74 65 72 6E 41 6C 20 43 41 52 5F 6F 74 30 1E 17 External CA Root0...
0008001C 01 22 30 06 03 29 2A 86 48 67 0F 01 01 01 05 .....*.....*H*P.....
0008001D 00 03 82 01 0F 00 82 01 0A 02 01 00 87 .....0.....0.....0
0008001E F7 1A 33 E2 02 04 0D 2B 39 1E 58 ED 1F BC 6C .....3..3e0...9..9..1
0008001F 0F 0C B5 7A 33 24 3E CE CE 5D 11 33 97 A4 29 7C 70 [...].....0.....0.....
[Icons] 15 Windows (ANSI) hex
```



Đây là kết quả sau khi bạn đã thực hiện các bước chỉnh sửa tệp tin: chương trình đã được khởi chạy thành công



Kết quả netstat hiển thị dòng TCP 0.0.0.0:4444 0.0.0.0:0 LISTENING cho thấy backdoor đã hoạt động thành công, mở cổng 4444 trên tất cả các địa chỉ IP, sẵn sàng chấp nhận kết nối từ bên ngoài.

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\admin>netstat -an | findstr 4444
    TCP        0.0.0.0:4444          0.0.0.0:0             LISTENING

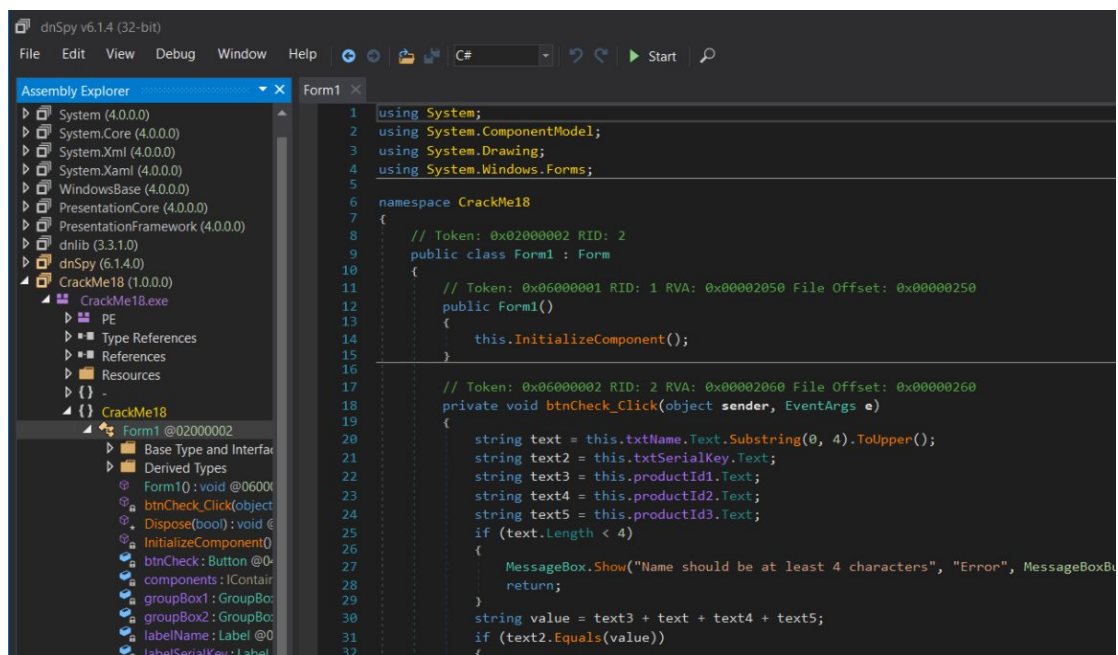
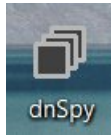
C:\Users\admin>netstat -an | findstr 4444

C:\Users\admin>
C:\Users\admin>
```

Bài tập Crackme18

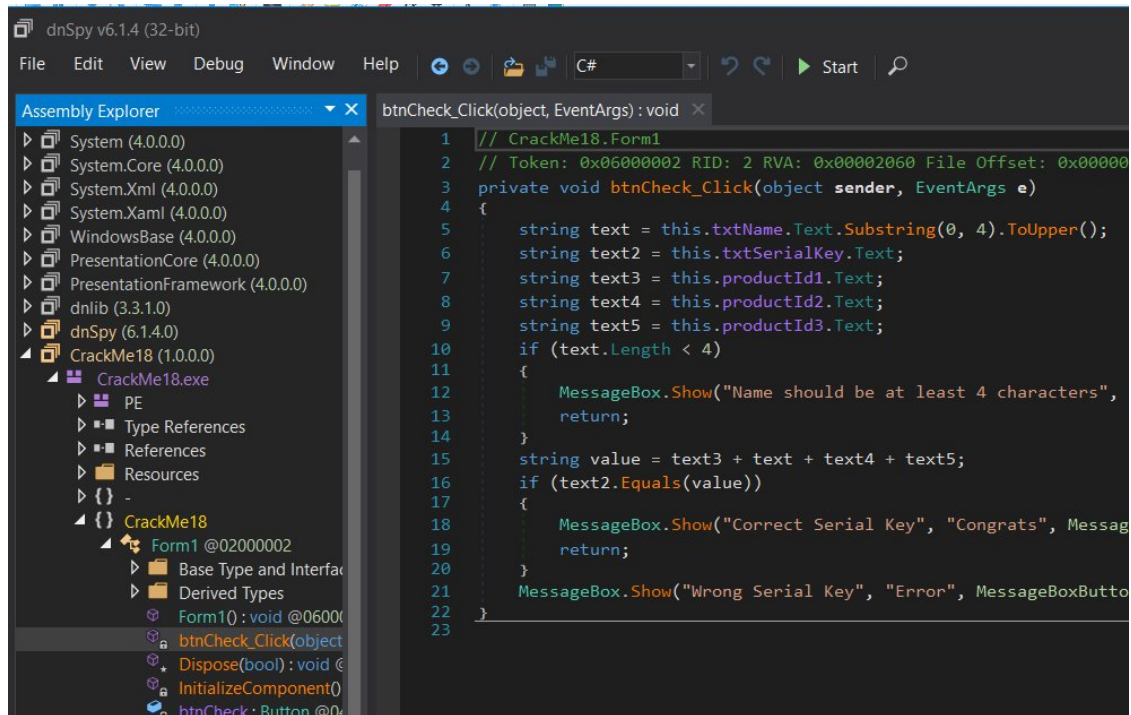
Bước 1:

- Em xác định chương trình được phát triển bằng C# và là tệp thực thi thuộc nền tảng .NET Framework.
- Em sử dụng công cụ dnSpy để decompile và phân tích mã nguồn của chương trình.
- Em mở tệp CrackMe 18 bằng dnSpy để tiến hành phân tích.
- Sau khi quá trình decompile hoàn tất, em có thể xem trực tiếp mã nguồn C# của chương trình, từ đó phục vụ cho việc phân tích và tìm hiểu cơ chế hoạt động



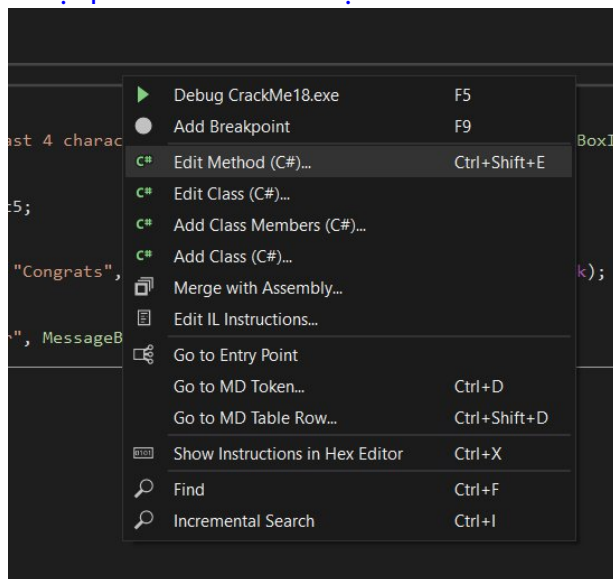
Bước 2:

- Em cần xác định chương trình xử lý như thế nào sau khi nhấn nút CHECK.
- Em chọn hàm btnCheck_Click để phân tích luồng xử lý của chương trình.



Bước 3:

- Em đọc đoạn mã và thấy chương trình gán Name vào biến text, Serial Key vào biến text2, còn text3, text4, text5 chứa các Product ID.
- Chương trình kiểm tra xem text có nhiều hơn 4 ký tự hay không.
- Nếu điều kiện đúng, chương trình lấy 4 ký tự đầu của text, chuyển sang chữ in hoa, ghép với các Product ID để tạo chuỗi.
- Cuối cùng, chương trình so sánh chuỗi vừa tạo với Serial Key mà người dùng nhập (text2).
- Để bỏ qua bước kiểm tra Serial Key, em chỉnh sửa phương thức bằng cách nhấp chuột phải vào hàm và chọn Edit Method.



Bước 4:

- Em chỉnh sửa dòng 10 và 16 như hình minh họa.

- Em thêm toán tử ! để đảo ngược điều kiện kiểm tra.
- Sau khi chỉnh sửa, chương trình không còn yêu cầu Name phải có ít nhất 4 ký tự.
- Đồng thời, chương trình sẽ chấp nhận mọi Serial Key được nhập và luôn hiển thị kết quả hợp lệ.

```

9      string text4 = this.productId2.Text;
10     string text5 = this.productId3.Text;
11     if (text.Length < 0)
12     {
13         MessageBox.Show("Name should be at least 4 characters", "Error");
14         return;
15     }
16     string value = text3 + text + text4 + text5;
17     if (!text2.Equals(value))
18     {
19         MessageBox.Show("Correct Serial Key", "Congrats", MessageBoxButtons.OK);
20         return;
21     }
22     MessageBox.Show("Wrong Serial Key", "Error", MessageBoxButtons.OK);
23 }

```

Bước 5:

- Sau khi hoàn tất chỉnh sửa, em chọn Save Module để lưu các thay đổi vào tệp thực thi.
- Em chạy lại chương trình và kiểm tra kết quả.
- Chương trình chấp nhận dữ liệu đã nhập, chứng tỏ em đã crack thành công Task 1

Hoàn thành bài crackme!

